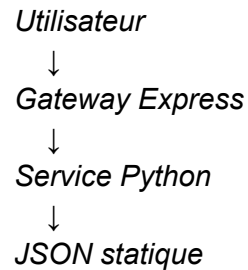


Architecture et flux de données du projet

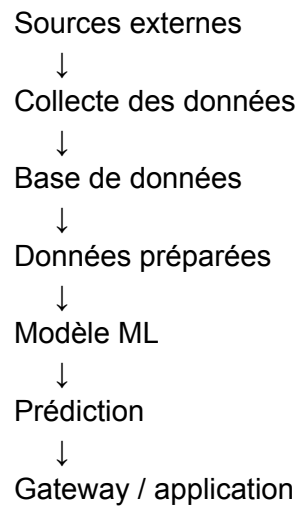
Le but général du brief

Aujourd'hui, EnergIA ressemble à ça :



Le problème : pour prédire la consommation électrique, le service Python ne peut pas se baser uniquement sur un JSON statique.

Il faut donc réfléchir à :



ÉTAPE 1 Choisir 4-6 variables

1. Temps

heure

jour de la semaine

week-end

mois

saison

vacances scolaires

jours fériés

2. Météo

température

humidité

ensoleillement

précipitations

Vent

3. Type de journée

jour ouvré

week-end

jour férié

Vacances

4. Région

région

population

densité

activité industrielle

5. Événements exceptionnels

canicule

grand froid

événement sportif

confinement, etc.

Choisir les données utiles



ÉTAPE 2 Trouver les vraies sources

Maintenant que vous avez choisi vos dimensions, vous devez répondre :

D'où viennent ces données ?

Par exemple :

Donnée	Source possible	Format	Fréquence
Consommation	RTE	CSV/API	horaire
Température	API météo	JSON	horaire
Ensoleillement	API météo	JSON	horaire
Vacances	API gouvernementale	JSON/CSV	ponctuel
Jours fériés	API calendrier	JSON	annuel
Heure/jour	calcul Python	—	temps réel

Vous devez identifier **de vraies sources**.

Et surtout, le brief demande :

au moins une source réelle testée

Donc vous devez faire au minimum un appel API ou télécharger un fichier pour montrer :

API

↓

réponse JSON

↓

donnée récupérée

↓

Ici, le brief vous demande de comprendre **comment les données arrivent**.

Il y a deux possibilités principales.

Étape 3 — Batch ou temps réel ?

Les données sont récupérées régulièrement par lots.

Exemple :

Tous les jours à 02:00

RTE



Script Python



Récupération données



Base de données

Exemple parfait :

historique de consommation

Tu n'as pas besoin de récupérer toute l'année de consommation à chaque prédiction.

Streaming / temps réel

La donnée est récupérée au moment où elle est nécessaire.

Exemple :

Utilisateur demande une prédiction



Gateway



Python



API météo



Température actuelle



Modèle ML

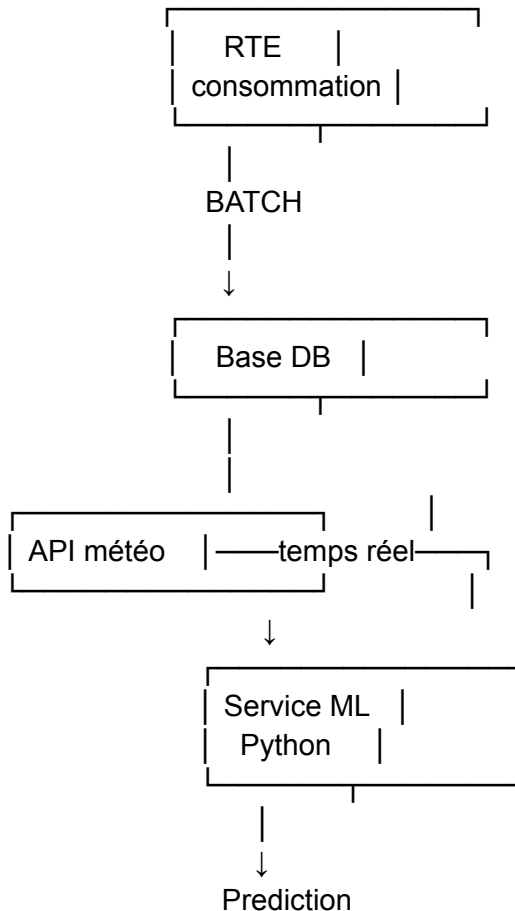


Prédiction

La météo actuelle peut être récupérée en temps réel.

Exemple de architecture

Vous pourriez avoir :



Étape 4 — Dessiner le flux de données

Ici, vous devez faire un **schéma d'architecture**.

Le brief vous pose plusieurs questions.

Question 1

Le module de prédiction est-il un nouveau microservice ?

Je vous conseillerais :

Gateway Express



Prediction Service



Model ML

Donc un **microservice Python séparé**.

Par exemple :

gateway/



prediction-service/

Pourquoi ?

Parce que le modèle ML est indépendant du Gateway.

Question 2

Comment transmettre la prédiction ?

Par exemple :

```
{  
  "prediction_mw": 4520,  
  "model_version": "v1.2",  
  "timestamp": "2026-08-09T13:00:00"  
}
```

Le service Python retourne cette réponse au Gateway.

Question 3

Que faire si l'API météo est indisponible ?

Il faut prévoir un **fallback**.

Par exemple :

API météo disponible



température réelle

↓
prédiction

Mais :

API météo indisponible

↓
utiliser dernière température connue
↓
prédiction

Ou retourner une erreur claire :

```
{  
  "error": "Weather service unavailable"  
}
```

Il faut simplement **choisir une stratégie et la justifier**.

Étape 5 — Concevoir la base de données

C'est une partie très importante.

Aujourd'hui :

JSON statique

Vous devez proposer quelque chose comme :

Database

- |
- |— consumption
- |— weather
- |— calendar
- |— prediction
- |— model

Par exemple :

consumption

colonne

exemple

id	1
timestamp	2026-08-09 13:00
region	Occitanie
consumption_mw	4520

weather

colonne	exemple
id	1
timestamp	2026-08-09 13:00
temperature	28.5
humidity	55
sunshine	80

prediction

colonne	exemple
id	1
timestamp	2026-08-09 13:00
predicted_mw	4600
model_version	v1.2
actual_mw	4520
mae	...

model

colonne	exemple
id	1
version	v1.2

trained_at 2026-08-01

model_path models/model_v1.2.pkl

Étape 6 — Prévoir les problèmes en production

Ici, le brief devient **MLOps**.

Il vous demande :

Que se passe-t-il quand le modèle est réellement utilisé ?

Vous devez répondre à plusieurs problèmes.

1. Où mettre les clés API ?

Jamais dans le code.

 Mauvais :

```
API_KEY = "123456789"
```

 Correct :

```
.env
```

Puis :

```
os.getenv("WEATHER_API_KEY")
```

Et `.env` doit être dans `.gitignore`.

2. Que faire si le modèle n'est pas entraîné ?

Vous devez choisir.

Par exemple :

Prediction request



Model disponible ?



OUI



NON



prediction



erreur 503

Je trouve cette solution préférable à inventer une prédiction.

3. Faut-il mettre les prédictions en cache ?

Oui, potentiellement.

Exemple :

Même demande



Prediction déjà calculée ?



OUI



retourner résultat

Cela évite de recalculer inutilement.

Étape 7 — MLOps : surveiller le modèle

C'est probablement la partie qui semble la plus compliquée, mais l'idée est simple.

Imagine que modèle fait aujourd'hui :

Prédiction : 4500 MW

Réel : 4520 MW

Erreur : 20 MW

Demain :

Prédiction : 4500

Réel : 4600

Erreur : 100

Puis :

Erreur moyenne

↓

20 MW

30 MW

45 MW

70 MW

100 MW

Le modèle devient moins performant.

Vous devez donc stocker le modèle utilisé

Par exemple :

prediction

id

timestamp

prediction

actual_value

model_version

Ainsi vous savez :

Prédiction 4520

réalisée avec

model v1.2

C'est ce que signifie :

traçabilité du modèle

MAE

Le brief parle de **MAE**.

MAE = **Mean Absolute Error**

En français :

erreur absolue moyenne.

Exemple :

Prédiction	Réalité	Erreur
4500	4520	20
4600	4550	50
4700	4650	50

MAE :

$$(20 + 50 + 50) / 3 \\ = 40 \text{ MW}$$

Donc :

Notre modèle fait en moyenne une erreur de 40 MW.

Drift

Le **drift**, c'est lorsque les données réelles changent avec le temps.

Par exemple :

2026
Modèle entraîné
↓
Habitudes normales
↓
Bonne prédiction

Puis :

2027
Nouvelles habitudes
+ véhicules électriques
+ nouveaux logements

+ changement climatique



Modèle moins performant

Donc :

Erreur augmente



détection



réentraînement

Réentraînement

Le brief vous demande également de discuter :

Option A — tous les mois

Chaque mois



réentraîner

Avantage : simple.

Inconvénient : on réentraîne même si le modèle fonctionne parfaitement.

Option B — selon la performance

$MAE < \text{seuil}$



ne rien faire

$MAE > \text{seuil}$



réentraîner

Avantage : plus intelligent.

Inconvénient : il faut surveiller les performances.

Pour projet, vous pouvez proposer :

réentraînement mensuel avec déclenchement anticipé si la MAE dépasse un seuil défini.

C'est une bonne réponse d'architecture.

Au final, qu'est-ce que vous devez rendre ?

Ton brief demande essentiellement **6 livrables**.

1 Catalogue des dimensions

Un tableau :

Dimension | Variable | Pourquoi | Priorité

avec **4 à 6 dimensions**.

2 Tableau des sources

Dimension

Source

Format

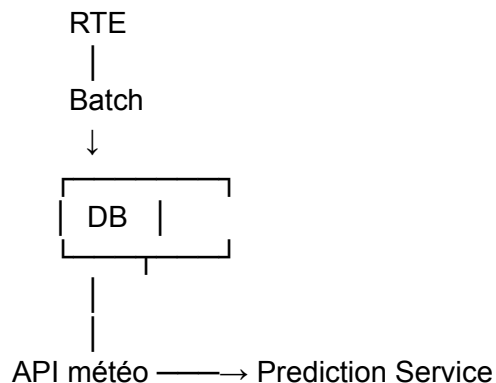
Fréquence

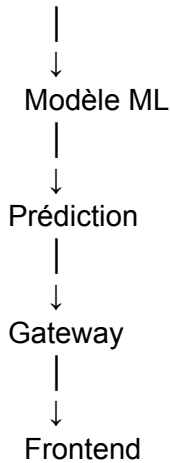
Accès

Batch / temps réel

3 Schéma du flux

Quelque chose comme :





4 Réponses d'architecture

Vous devez répondre :

- microservice séparé ou non ?
- comment envoyer la prédiction ?
- que faire si météo indisponible ?
- que faire si modèle indisponible ?
- cache ou non ?
- où mettre les clés API ?

5 Schéma de base de données

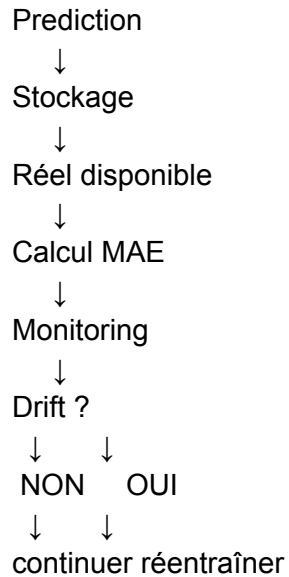
Par exemple :

consumption
weather
calendar
prediction
model

avec les relations entre elles.

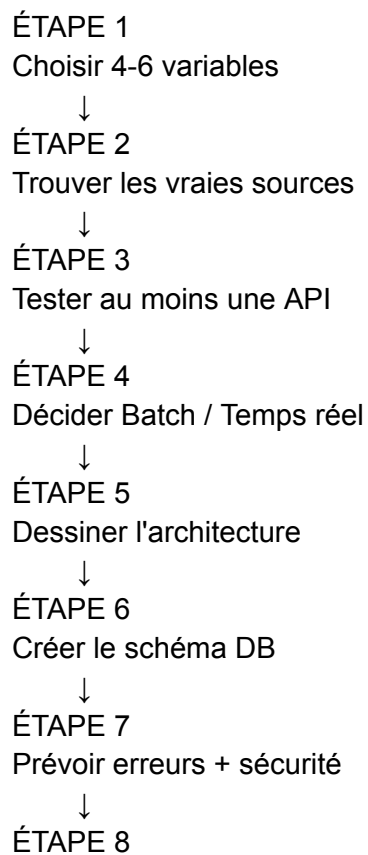
6 MLOps

Vous devez expliquer :



En résumé : travail étape par étape

Je te conseille de travailler exactement dans cet ordre :



Prévoir monitoring / MAE / drift



ÉTAPE 9

Documenter vos choix

Important : brief ne demande pas encore de construire tout le modèle ML. Il vous demande d'abord de **réfléchir à l'architecture des données qui permettra ensuite au modèle ML de fonctionner correctement.**